Extending Distribution-Independent Estimators for the Traveling Salesman Problem to Arbitrary Dimensions

Stephen Feldman, James P. Bailey, Bahar Çavdar
Rensselaer Polytechnic Institute, Troy, NY
{feldms5, bailej6, cavdab2}@rpi.edu

Abstract

We present **GART 2.0**, a distribution-independent, dimension-agnostic estimator of optimal TSP tour length. Existing estimators either depend on convex-hull or bounding-box geometry—which is intractable for $d \geq 3$—or require the input distribution as a known model parameter. GART 2.0 does neither: it combines a deterministic Minimum Spanning Tree with a LightGBM regression on sample-statistic features (coordinate-range, centroid, and MST-topology descriptors) to predict the scaling ratio $\alpha = L_{\text{TSP}}/L_{\text{MST}}$. Because α is a bounded, unit-less ratio and the features are dimension-agnostic, the same trained model applies to $d = 2$, to $d = 100$, and—via classical multidimensional scaling on the distance matrix—to non-Euclidean metrics. The estimator's primary purpose is *precision*: small, predictable deviation from optimality is a prerequisite for using tour-length estimates inside larger optimization pipelines, where an imprecise estimate cannot be recalibrated.

Keywords Traveling Salesman Problem · Tour cost estimation · Distribution-independent · Multidimensional networks

1 Introduction

The Traveling Salesman Problem (TSP) is a fundamental model in combinatorial optimization, widely used for logistics operations planning, network design, and resource allocation [Chien, 1992, Applegate et al., 2006]. Because determining exact solutions for large-scale instances is NP-hard [Applegate et al., 2006], the ability to compute rapid tour-cost estimates is essential for real-time decision-making [Held and Karp, 1962]. Tour-cost estimates also serve as inputs to higher-level optimization problems where the TSP appears as a subproblem. For example, in the mobile facility location problem [Halper and Raghavan, 2011], the objective function includes the cost of a TSP route through customer locations; for $n \geq 100$, replacing the exact TSP solve with a fast estimate reduces computation by one to three orders of magnitude.

1.1 Background and Motivation

Tour-length estimation is a key input to strategic logistics decisions where solving the TSP exactly is unnecessary or infeasible. Last-mile delivery accounts for approximately 53% of total shipping costs [Finmile, 2025], and the UPS ORION system saves 100 million miles and $400 million annually through route optimization [Holland et al., 2017]. In such systems, rapid cost estimates—rather than exact solutions—drive decisions like fleet sizing, territory partitioning, and service-level pricing. For example, a fleet planner must estimate the total route cost across hundreds of delivery zones to allocate vehicles; solving each zone's TSP exactly would take hours, while a tour-length estimator provides answers in milliseconds.

Beyond traditional logistics, fast TSP length approximation is a critical component across diverse applications in which TSP-like structures arise as subproblems. One of these application areas is bioinformatics. The ordering of markers in radiation hybrid mapping is modeled as a TSP, where rapid cost estimation enables evaluation of assembly quality without full reconstruction [Agarwala et al., 2000]. In data compression, vector quantization partitions a codebook into clusters and must order codewords to minimize transition costs, a problem reducible to TSP on a high-dimensional distance matrix. Additionally, in sensor network design, estimating the minimum data collection path length is crucial for optimizing the lifespan of battery-constrained mobile sinks in wireless sensor networks [Wang et al., 2008].

In these domains, the problem is rarely confined to 2D Euclidean space. Genome assembly operates in abstract metric spaces defined by overlap similarity [Agarwala et al., 2000], while sensor networks involve routing through high-dimensional feature spaces [Wang et al., 2008]. While a full evaluation on these application-specific metrics is beyond the scope of this study, they motivate the need for an estimator that generalizes beyond 2D Euclidean geometry. Furthermore, metaheuristics for large-scale routing frequently partition problems into smaller sub-TSPs, each requiring a rapid cost estimate to guide the search [Mariescu-Istodor and Fränti, 2021, Pan et al., 2023]. For example, Pan et al. [2023] decompose large TSP instances hierarchically, solving each cluster as an independent sub-problem. A fast, accurate tour-length estimator enables such algorithms to evaluate candidate partitions without solving each sub-TSP exactly.

Despite this breadth of applications, existing TSP tour-length estimators share a critical dependence on geometric enclosure metrics—specifically the convex hull area or bounding box volume—that fail in two important ways. First, convex hull computation scales as $O(n^{\lfloor d/2 \rfloor})$ [Preparata and Hong, 1977], making it intractable for $d > 3$. Second, area-based formulas produce degenerate estimates on non-uniform distributions: when points lie along a diagonal line, the convex hull area approaches zero and estimators predict a near-zero tour cost, despite the true tour remaining long. Our approach addresses both limitations by replacing convex hull enclosure with a combination of MST topology and lightweight bounding-box statistics. The MST captures outlier structure and cluster separation directly from edge weights, while coordinate-range features (hypervolume, aspect ratio, centroid distances) provide scale information without requiring hull computation. We demonstrate that this approach yields lower error than all existing estimators on both synthetic and real-world TSPLIB95 benchmarks.

1.2 Limitations of the Current Models

While the need for a TSP estimation model for general cases is clear, a persistent gap remains in the literature regarding high-dimensional or non-Euclidean instances. Traditional estimators, such as the BHH theorem [Beardwood et al., 1959] and the probabilistic model by Chien [1992], were developed primarily for 2D Euclidean space. Moreover, these models commonly rely on the convex hull [Preparata and Hong, 1977] to estimate the "volume" or "area" of a point set. However, convex hull computation scales as $O(n^{\lfloor d/2 \rfloor})$, making it prohibitive beyond three dimensions [O'Rourke, 1998]. This limitation also affects non-Euclidean metric spaces: while Menger [1928] showed that any metric space satisfying the triangle inequality can be embedded into a high-dimensional Euclidean space, doing so only shifts the convex hull problem to higher dimensions rather than resolving it.

Recent work has attempted to escape these constraints with neural networks. Varol et al. [2024] report sub-1% deviation on standard distributions by combining an MLP with domain-knowledge features, but their pipeline requires solution-level features computed from heuristic tours—presupposing access to a solver. Kou et al. [2022] use coordinate standard deviation as a single-feature predictor but remain restricted to 2D Euclidean instances. Neither addresses both the dimensional and the solver-free constraints simultaneously; GART 2.0 targets exactly that gap.

1.3 Contribution: The GART 2.0 Model

Throughout this work, "estimation" refers to predicting the scalar tour cost without constructing a tour, distinguishing GART 2.0 from construction heuristics and approximation algorithms that produce feasible tours with worst-case ratio guarantees. We use “distribution-free” in the sense of Cavdar and Sokol [2015]: the estimator does not require prior knowledge of the vertex distribution as an input; this does not imply uniform accuracy across all distributions (Section 5 discusses cases where error increases).

Our work contributes to the literature on distribution-independent estimators, a tradition first formalized as "distribution-free" in the 2D regression model of Cavdar and Sokol [2015]. We keep the Cavdar–Sokol design principle—estimate from sample statistics rather than from a declared distribution parameter—but replace features that scale poorly with dimensionality (convex hulls, $O(n^{\lfloor d/2 \rfloor})$; PCA [Jolliffe, 2002]) with MST-derived topological statistics and lightweight coordinate-range descriptors computable in $O(n \cdot d)$.

The simplest practical baseline for TSP estimation is the *fixed MST ratio*: $\hat{L} = \rho(d) \cdot L_{\text{MST}}$, where $\rho(d)$ is a dimension-dependent constant calibrated from empirical data (e.g., $\rho(2) \approx 1.075$ for uniform random instances [Percus and Martin, 1996]); see Section 5.1.5 for the formal definition. This approach is fast, requires no geometric features, and works in any dimension. However, it applies a single global constant regardless of the instance's topological structure—clustered instances, skewed distributions, and geometric lattices all receive the same multiplier. On the 78 EUC_2D TSPLIB95 instances, this limitation produces 5.22% MAPE and 4.54% SDPE. On the diverse 2D synthetic suite described in Section 5.2, the fixed ratio's error rises to 12.45% MAPE with SDPE = 11.06% — inconsistent as well as inaccurate.

The primary contribution of this work is the Geometrically Assisted Regression Tree (GART 2.0) model, a LightGBM-based framework [Ke et al., 2017] that learns to predict a *per-instance* scaling factor α rather than using a fixed constant. By combining MST-centric features—such as the leaf ratio, dominance ratio, and normalized diameter—with lightweight geometric descriptors (bounding hypervolume, centroid statistics, aspect ratio), GART 2.0 adapts its estimate to the topological structure of each instance. We frame GART 2.0 as a *precision-first* estimator: accuracy (low MAPE) is desirable, but *precision*—the consistency of predictions as measured by the Standard Deviation of Percentage Error (SDPE)—is the more fundamental contribution. A precise estimator with a small systematic offset can be recalibrated through a single scaling constant; an imprecise estimator cannot be corrected post hoc. On 2D synthetic data, GART 2.0 achieves SDPE = 4.94% vs. 11.05% for the MST Ratio (2.2$\times$ tighter, Levene's test $p < 10^{-127}$); on multi-dimensional data, SDPE = 1.17% vs. 6.89% (5.9$\times$ tighter, $p \approx 0$). Crucially, GART 2.0 retains the computational advantage of MST-based estimation: prediction requires only the MST (computable in $O(n \log n)$ for 2D via Delaunay triangulation, $O(n^2 \cdot d)$ in general) plus a constant-time LightGBM inference step, yielding sub-second predictions for instances where exact solvers require minutes to hours.

1.4 Paper Outline

The remainder of this paper is structured as follows. Section 2 introduces the theoretical foundations of the multiplicative MST–TSP relationship and the Christofides bound [Christofides, 1976]. Section 3 describes GART 2.0's feature set, dataset generation, and implementation. Section 4 presents model training with hyperparameter tuning via Optuna [Akiba et al., 2019] and complexity analysis. Section 5 defines the baseline estimators, introduces the three benchmark datasets, reports per-dataset results, and discusses the patterns they reveal. Section 6 extends GART 2.0 to non-Euclidean TSPLIB95 instances via MDS embedding. Section 7 summarizes contributions, limitations, and future directions.

2 Theoretical Foundations

To formally situate the estimation problem, we first define the Traveling Salesman Problem (TSP) using its classic integer linear programming (ILP) formulation. Let $G = (V, E)$ be a complete undirected graph where $V = \{1, \dots, n\}$ is the set of vertices and $E = \{(i, j) : i, j \in V, i < j\}$ is the set of edges. Associated with each edge (i, j) is a non-negative cost c_{ij}, typically representing the Euclidean distance between points in $\mathbb{R}^d$. The objective is to find a tour of minimum total cost that visits every vertex exactly once. Following the standard Dantzig-Fulkerson-Johnson formulation found in Applegate et al. [2006], we use binary decision variables x_{ij}, where $x_{ij} = 1$ if edge (i, j) is included in the tour, and 0 otherwise. The problem formulation is as follows:

$$\min \sum_{i=1}^{n} \sum_{j=i+1}^{n} c_{ij} x_{ij} \tag{1}$$

Subject to the degree constraints:

$$\sum_{j<i} x_{ji} + \sum_{j>i} x_{ij} = 2, \quad \forall i \in V \tag{2}$$

And the subtour elimination constraints:

$$\sum_{i \in S} \sum_{j \in S, j>i} x_{ij} \leq |S| - 1, \quad \forall S \subset V, 2 \leq |S| \leq n-1 \tag{3}$$

The TSP is NP-hard [Karp, 1972]. This formulation establishes the combinatorial structure we exploit: any feasible tour is a 2-regular spanning subgraph, of which the MST is the cheapest connected relaxation.

In contrast to the intractable TSP, the Minimum Spanning Tree (MST) problem, which seeks a connected, acyclic subgraph minimizing the total edge weight, can be solved optimally in polynomial time. Classical algorithms such as Prim's [Prim, 1957] and Kruskal's construct the MST in $O(n^2)$ time for complete graphs. The pairwise distance computation scales as $O(n^2 \cdot d)$, making the total cost linear in d and thus a computationally stable baseline for high-dimensional inference.

The relationship between these two problems provides a theoretical bound on the optimal TSP tour cost, denoted L_{TSP}. Since any valid TSP tour is a connected graph spanning all vertices, removing one edge yields a spanning tree. Thus, the weight of the MST, L_{MST}, is a strict lower bound. Conversely, a valid tour can be constructed by traversing the MST edges twice (a depth-first walk) and applying shortcuts via the triangle inequality, establishing the classic double-tree bound:

$$L_{\text{MST}} \leq L_{\text{TSP}} \leq 2 \cdot L_{\text{MST}}. \tag{4}$$

Christofides [1976] improved this upper bound by augmenting the MST with a Minimum Weight Perfect Matching (MWPM) on the set of vertices with odd degrees. This yields a tour whose length is guaranteed to be at most $\frac{3}{2} L^*_{\text{TSP}}$, where L^*_{TSP} denotes the optimal tour cost, in metric spaces. This $\frac{3}{2}$-approximation ratio was recently improved to $(\frac{3}{2} - \epsilon)$ by Karlin et al. [2021], though the practical improvement is negligible. However, calculating the MWPM requires $O(n^3)$ time, which is significantly more expensive than the $O(n^2)$ MST construction. For large-scale auxiliary subproblems where computation speed is critical, the cubic complexity of the Christofides refinement is often prohibitive, necessitating a faster method to approximate the tightening coefficient.

Leveraging the relationship between the optimal TSP and MST values, instead of explicitly constructing the matching, we treat the tightening coefficient as a statistical parameter $\alpha = L_{\text{TSP}}/L_{\text{MST}}$. While the worst-case bound for α from the double-tree argument is 2.0 (Equation 4), the Christofides heuristic empirically tightens practical values. The expected value of α depends on both the dimensionality d and the instance size n. Percus and Martin [1996] showed that for uniformly distributed points, α decreases as d increases: the MST and TSP lengths converge because higher-dimensional spaces offer more "shortcuts" for the tour to exploit relative to the tree. This implies that the computational gap between the MST and the optimal TSP tour diminishes as spatial freedom increases. Consequently, a static estimator calibrated for 2D (where $\alpha \approx 1.075$ for uniform random instances [Percus and Martin, 1996]) will systematically overestimate tour costs in higher dimensions. Our proposed model addresses this by learning to predict α dynamically based on the topological characteristics of the vertex set.

3 Methodology: The Geometrically Assisted Regression Tree

We propose the Geometrically Assisted Regression Tree (GART 2.0), a framework that estimates TSP tour cost using the relationship between the MST and the optimal tour. GART 2.0 integrates gradient-boosted regression trees with a dimension-agnostic feature set derived from the MST topology to overcome the dimensional limitations of classical geometric estimators. Unlike traditional approaches that map coordinate-dependent metrics directly to tour cost, GART 2.0 decouples the estimation into two phases: (1) the deterministic computation of the MST, and (2) the statistical prediction of the scaling coefficient $\alpha = L_{\text{TSP}}/L_{\text{MST}}$. The final estimator is $\hat{L}_{\text{TSP}} = \hat{\alpha} \cdot L_{\text{MST}}$, where $\hat{\alpha}$ is predicted by a LightGBM regressor based on 29 topological features of the instance.

Target bounds. As a design decision we constrain the regression target to $\alpha \in [1.0, 2.0]$. The lower bound is feasibility-exact: every TSP tour is a spanning structure, so $L_{\text{TSP}} \geq L_{\text{MST}}$. The upper bound is the double-MST worst-case ratio, and in all training data we observe α well inside this interval. Predictions are therefore clipped to $[1.0, 2.0]$ at inference; this acts only as a safety clamp for pathological instances where the MST approximation is degenerate (the regime outside the model's assumed operating conditions). The following section introduces the feature set.

3.1 Feature Selection

To predict the scaling factor α accurately across varying dimensions, we extract 29 features from each TSP instance. The feature count reflects the need to characterize three distinct aspects of instance structure—spatial spread, edge weight distribution, and branching topology—each requiring multiple statistics to capture the range of behaviors observed across distributions. While prior estimators such as Cavdar and Sokol [2015] use coordinate-based statistics tied to specific axes (e.g., standard deviations of x and y positions) and convex hull area, our features combine MST edge weights and degree sequence—which are invariant to rotation, translation, and coordinate system choice—with lightweight geometric descriptors (bounding hypervolume, centroid distances) that generalize naturally to arbitrary d. The dimensionality d enters the feature set as an explicit input rather than implicitly through coordinate geometry.

All features can be computed in $O(n^2 \cdot d)$ time or less, dominated by the pairwise distance matrix required for MST construction. No feature requires convex hull computation or geometric partitioning; the geometric descriptors use coordinate ranges computable in $O(n \cdot d)$. We organize the features into three groups: (1) geometric and centroid descriptors, which quantify spatial spread and density; (2) MST edge distribution statistics, which capture local uniformity through edge weight moments; and (3) topological structure and clustering proxies, which detect higher-order patterns such as clusters and filaments.

3.1.1 Geometric and Centroid Descriptors

The first category quantifies the macroscopic spatial properties of the node set in $\mathbb{R}^d$. These features provide the model with scale information analogous to the volume term V in the BHH formula $\hat{L} = \beta_d \cdot n^{(d-1)/d} \cdot V^{1/d}$ [Beardwood et al., 1959], but computed from coordinate ranges rather than convex hull volume. Centroid distance statistics, adapted from Cavdar and Sokol [2015], measure the dispersion of nodes relative to the geometric center, enabling the model to distinguish uniform distributions from centralized Gaussian clusters. Table 1 lists these features.

3.1.2 MST Edge Distribution Statistics

The second category captures the local uniformity of the point set through the statistical moments of the MST edge weights. Unlike coordinate-based variance, edge weights are invariant to rotation and translation. High skewness or kurtosis in this distribution typically indicates a mixture of scales, such as dense clusters separated by voids, which significantly impacts the α ratio.

Table 1: Geometric and Centroid Feature Specifications.

Feature Name	Description & Rationale	Complexity	Origin / Citation
Dimension (d)	The number of spatial coordinates defining each point.	$O(1)$	Standard
Number of Customers (n)	The total count of vertices in the instance.	$O(1)$	Standard
Aspect Ratio	Ratio of the largest dimension range to the smallest. Detects subspace distortion.	$O(n \cdot d)$	**Proposed**
Bounding Hypervolume	Product of coordinate ranges: $\prod(\max_d - \min_d)$. Approximates volume V for BHH bounds.	$O(n \cdot d)$	Adapted from Beardwood et al. [1959]
Node Density	Ratio of n to Bounding Hypervolume. Proxy for point intensity $\delta = n/V$.	$O(1)$	**Proposed**; cf. Beardwood et al. [1959]
Centroid Dist (Mean, Std)	Average and deviation of distances to the centroid. Measures central tendency.	$O(n \cdot d)$	Adapted from Cavdar and Sokol [2015]
Centroid Dist Max	Radius of the enclosing sphere.	$O(n \cdot d)$	Adapted from Cavdar and Sokol [2015]
Centroid Dist IQR	Interquartile range of centroid distances. Robust dispersion metric.	$O(n \cdot d)$	**Proposed**; extends Cavdar and Sokol [2015]

Table 2: MST Edge Statistic Specifications

Feature Name	Description & Rationale	Complexity	Origin / Citation
MST Edge Max	Weight of the longest edge in the MST.	$O(n)$	Standard
MST Quantiles	10%, 25%, 50%, 75%, 90% percentiles. Provides a non-parametric distribution profile.	$O(n \log n)$	**Proposed**; via MST [Prim, 1957]
MST Total Length	Sum of all edge weights. The primary scaling factor; $L_{\text{TSP}} \geq L_{\text{MST}}$.	$O(n^2)$	Prim [1957]
MST Edge Mean	Average edge length in the MST.	$O(n)$	Smith-Miles and van Hemert [2010]
MST Edge Std	Standard deviation of edge lengths in the MST.	$O(n)$	Smith-Miles and van Hemert [2010]
MST Edge Skewness	Asymmetry of edge distribution. High positive skew implies many short edges and few long bridges.	$O(n)$	Smith-Miles and van Hemert [2010]
MST Edge Kurtosis	Tailedness of distribution. Indicates the presence of extreme outliers.	$O(n)$	Smith-Miles and van Hemert [2010]

3.1.3 Topological Structure and Clustering Proxies

The final category contains features designed to detect higher-order structures—such as clusters, filaments, and hubs—without performing expensive clustering algorithms. We introduce specific ratios (Dominance, Gap, Normalized Diameter) that serve as $O(n)$ proxies for these complex geometric properties. For instance, the "Leaf Ratio" effectively distinguishes between low-dimensional linear manifolds (low leaf count) and high-dimensional isotropic noise (high leaf count).

3.2 Dataset Generation

To train a general model, we constructed a synthetic dataset with topological diversity across four distribution classes: Uniform, Normal (Gaussian), Clustered (k-means centers), and Correlated (autoregressive across dimensions). For each instance, dimensions are independently assigned a distribution class, producing anisotropic configurations where the topology varies by axis. This “mix-and-match” strategy avoids bias toward any single geometric type. Random seeds are derived from distribution signatures to ensure distinct realizations across instances.

The training dataset comprises 51,818 instances with node counts $n \in [20, 1000]$ and dimensions $d \in \{2, 3, 4, 5, 6, 7, 8, 9, 10, 15, 20, 25, 30, 35, 40, 45, 50\}$ (17 dimension values), split into training (37,283), validation (9,286), and test (5,249) partitions. Solved instances exist for d up to 100 (see Appendix B); extrapolation beyond the training range ($d = 100$) is evaluated in Section 5.3.1.

For each instance, the best-known tour cost was computed by running both Concorde [Applegate et al., 2006] and LKH-3 [Helsgaun, 2017], retaining the shorter result as ground truth. Concorde provides provably optimal solutions for

Table 3: Topological and Clustering Feature Specifications

Feature Name	Description & Rationale	Complexity	Origin / Citation
MST Dominance Ratio	Sum of largest $\sqrt{n}$ edges / Total Length. Captures weight of inter-cluster bridges.	$O(n)$	**Proposed**; via MST [Prim, 1957]
MST Gap Ratio	Max edge / Median edge. Measures magnitude of sparsity jumps.	$O(1)$	**Proposed**; via MST [Prim, 1957]
MST Leaf Ratio	Fraction of nodes with degree $k = 1$. Topology proxy for dimensionality.	$O(n)$	Smith-Miles and van Hemert [2010]
MST Degree (Mean, Std)	Branching factor statistics.	$O(n)$	Smith-Miles and van Hemert [2010]
MST Degree Max	Maximum node degree. Identifies "hub" nodes common in High-D spaces.	$O(n)$	Smith-Miles and van Hemert [2010]
MST Diameter	Sum of edge weights on the longest path between any two leaves, computed via double-BFS.	$O(n)$	Graph Theory
Normalized Diameter	Diameter / Total Length. Scale-invariant shape proxy ($= 1.0$ for path-manifolds exactly).	$O(1)$	**Proposed**; via double-BFS
Large Edge Count	Count of edges $> \mu + \sigma$. Secondary cluster indicator.	$O(n)$	**Proposed**; via MST [Prim, 1957]

instances within its computational budget; LKH-3 is a state-of-the-art heuristic that empirically matches or approaches the optimum. For all training instances ($n \leq 1000$), Concorde confirmed optimality within its computational budget; LKH-3 matched or improved all Concorde solutions. Coordinate scaling was applied to preserve floating-point precision in the integer-valued TSPLIB format required by these solvers.

3.3 Implementation

The GART 2.0 pipeline is implemented in Python using SciPy [Virtanen et al., 2020] for Delaunay triangulation [Delaunay, 1934] and MST construction (via `scipy.sparse.csgraph.minimum_spanning_tree`), Numba [Lam et al., 2015] for JIT-compiled centroid distance computation, and scikit-learn [Pedregosa et al., 2011] for classical MDS embedding of non-Euclidean distance matrices. For instances without Euclidean coordinates (e.g., TSPLIB GEO and EXPLICIT types), MDS projects the distance matrix into 2D coordinates for geometric feature extraction while MST features are computed from the original distances; details are given in Appendix C and evaluated in Section 6. The LightGBM model [Ke et al., 2017] is trained and served via the Python API. All benchmarks were run on a single machine (AMD Ryzen 9 5900X, 64 GB RAM, Windows 11) with no GPU. Concorde version 03.12.19 and LKH-3 version 3.0.7 were used for ground-truth computation.

4 Estimation Model Development and Complexity Analysis

The relationship between topological features and the scaling factor α is non-smooth: sharp regime transitions occur, for example, when a point cloud transitions from a single cluster to multiple separated groups. This motivates a model architecture capable of capturing discontinuous, high-order feature interactions. GART 2.0 is implemented using the LightGBM gradient-boosting framework [Ke et al., 2017]. For the task of scalar regression on tabular topological features, Gradient Boosted Decision Trees (GBDTs) offer superior sample efficiency and faster inference compared to deep learning alternatives.

4.1 Model Architecture and Training

We selected LightGBM over standard Random Forests or traditional GBDT implementations due to its “leaf-wise” tree growth strategy. Unlike level-wise algorithms that grow trees depth-first to maintain balance, LightGBM splits the leaf with the maximum delta loss, regardless of depth. This approach is mathematically advantageous for approximating the α ratio because the relationship between graph topology and tour cost often exhibits sharp, non-smooth phase transitions—for example, the sudden drop in α when a random point cloud begins to form distinct clusters. Leaf-wise growth allows the model to allocate more complexity to these critical transition regions without wasting splits on simpler, linear parts of the manifold.

At inference time, the predicted $\hat{\alpha}$ is clipped to the theoretical interval $[1.0, 2.0]$ from Equation 4; in practice, none of the 16,907 predictions on the multi-dimensional test set fall outside this range (observed range $[1.031, 1.948]$), so clipping has no effect on reported accuracy.

The model was trained to minimize the Root Mean Squared Error (RMSE) between the predicted ratio $\hat{\alpha}$ and the ground truth α. To optimize the model's generalization capability, we employed the Optuna framework [Akiba et al., 2019] to perform a Bayesian search of the hyperparameter space. The optimization utilized the Tree-structured Parzen Estimator (TPE) algorithm over 100 trials to tune the learning rate, tree structure, and regularization terms. The final model configuration, detailed in Table 4, resulted in a robust ensemble of 4,431 trees. While the hyperparameter search allowed for up to 300 leaves per tree to capture complex interactions, the final trained model exhibited a maximum leaf count of 258, indicating that the constraint was sufficient to prevent unbridled overfitting while allowing necessary depth.

Table 4: GART 2.0 hyperparameters (Optuna TPE, 100 trials, RMSE objective) and trained-model statistics. Values match the committed `lgbm_model_v3/best_params_v3.json` and the saved `lgbm_alpha_model_v3.joblib` booster.

Parameter	Value
Tuned hyperparameters	
learning rate	0.0129
num leaves	142
L1 regularisation	8.49×10^{-8}
L2 regularisation	1.43
feature fraction	0.657
bagging fraction	0.432
bagging frequency	5
min child samples	22
Fixed / derived	
boosting type	gbdt
objective	regression_l2 (RMSE)
early stopping patience	100
random seed	42
trees (after early stop)	$4{,}431$
avg leaves per tree	144.5
max leaves (any tree)	258
avg tree depth	16.30
max tree depth	29

4.2 Feature importance and design validation

To verify that the 29-feature set is doing real work—rather than carrying redundant descriptors—we compute SHAP values [Lundberg and Lee, 2017] on the held-out test split using the saved booster (see `shap_analyzer_v3.py`). Table 5 reports the top ten features by mean absolute SHAP contribution. Three observations motivate the final feature set:

1. **MST topology dominates.** The MST dominance ratio alone accounts for roughly one quarter of the model's decision mass, and four of the top ten features are MST-derived. This is consistent with the BHH intuition that tour length scales with tree-like spanning structure; it also justifies the decision to feature-engineer from the MST rather than from raw coordinates.

2. **Dimension and size are first-class inputs.** n and d together contribute roughly 21% of the decision mass, which is how the model adapts to varying problem scale without being given a distribution label.
3. **Lightweight spread statistics add coverage.** Centroid-distance dispersion and bounding hypervolume contribute meaningfully without requiring any convex-hull or PCA computation, which is what allows the feature set to scale to arbitrary d.

Earlier design iterations carried a larger candidate pool; the reported 29-feature set is the pruned subset retained after SHAP-guided removal of redundant descriptors. We emphasize that SHAP is used here only to audit the *design*; the numerical results reported later are generated by the trained model with no further feature selection.

Table 5: Top-10 features of GART 2.0 by mean |SHAP|, computed on the held-out test split ($N = 5{,}000$ sampled rows, seed 42) with the trained booster. Share (%) is the share of total mean absolute SHAP mass.

Rank	Feature	Mean |SHAP|	Share (%)
1	`mst_dominance_ratio`	0.0319	23.2
2	`n_customers`	0.0161	11.7
3	`dimension`	0.0132	9.6
4	`mst_degree_mean`	0.0079	5.7
5	`centroid_dist_std`	0.0068	4.9
6	`bounding_hypervolume`	0.0061	4.4
7	`mst_diameter`	0.0057	4.1
8	`mst_total_length`	0.0050	3.7
9	`large_edge_count`	0.0049	3.6
10	`mst_diameter_normalized`	0.0049	3.5

4.3 Computational Complexity Analysis

A primary motivation for GART 2.0 is to provide high-quality estimates faster than constructive heuristics. To validate this, we analyze the worst-case time complexity of the GART 2.0 pipeline compared to the Christofides algorithm, which provides the tightest polynomial-time approximation guarantee ($\frac{3}{2} \cdot L^*_{\text{TSP}}$). The total runtime of GART 2.0 is the sum of the feature extraction time and the inference time.

The computational pipeline begins with the construction of the MST. For a complete graph with n vertices in d dimensions, calculating the pairwise distance matrix requires $O(n^2 \cdot d)$ operations. Constructing the MST from the dense matrix takes $O(n^2)$ time. For 2D instances, we exploit the fact that the MST is a subgraph of the Delaunay triangulation [Delaunay, 1934], which has $O(n)$ edges, reducing the MST construction to $O(n \log n)$. Extracting topological features (degrees, diameter, leaf counts) requires $O(n)$ time. Sorting the $n-1$ edges for quantile computation adds $O(n \log n)$. Therefore, feature extraction is dominated by the distance matrix: $O(n^2 \cdot d)$ in general, or $O(n \log n)$ for 2D.

The inference complexity of the LightGBM model is determined by the total number of split comparisons across the ensemble. Our trained model has $K = 4{,}431$ trees with a mean of 144.5 leaves per tree (maximum 258). LightGBM grows trees leaf-wise rather than level-wise [Ke et al., 2017], so depth is *not* $\lceil \log_2 L \rceil$; the actual average root-to-leaf depth is $\bar{D} \approx 16.3$ with a worst-case depth of 29 empirically. Inference therefore performs $K \cdot \bar{D} \approx 7.2 \times 10^4$ comparisons per prediction, independent of n. Including feature computation, the total asymptotic complexity of GART 2.0 is $O(n^2 \cdot d)$ in general and $O(n \log n)$ in 2D via the Delaunay-sparse MST construction described below.

In contrast, the Christofides heuristic requires a Minimum Weight Perfect Matching (MWPM) on odd-degree MST vertices. Exact MWPM algorithms run in $O(n^3)$ time [Cook and Rohe, 1999]. Approximate matching algorithms [Duan and Pettie, 2014, Drake and Hougardy, 2003] reduce this cost but sacrifice the 1.5-approximation guarantee, degrading the worst-case ratio to $1.5 + \epsilon$.

As noted in Section 2, Christofides provides a worst-case *upper bound*, not an estimate. GART 2.0 provides a statistical *point estimate* calibrated to minimize prediction error, aligning more closely with operational needs such as fleet sizing where the true cost is typically $1.08 \times L_{\text{MST}}$ [Percus and Martin, 1996].

5 Benchmarking

We evaluate GART 2.0 against established estimators on three benchmarks: a synthetic 2D dataset spanning five distribution classes, a multi-dimensional synthetic dataset covering $d \in \{2, \ldots, 100\}$ (with $d = 100$ reserved as an extrapolation bucket outside the training range), and the 78 EUC_2D instances of TSPLIB95 [Reinelt, 1991]. The 2D suite mixes uniform and non-uniform generators: anisotropic clusters, geometric lattices, and near-linear manifolds. The near-linear generator was not part of the training distribution; we include it explicitly to probe how MST-based estimation handles geometries outside what the model was taught.

Throughout this evaluation, we report **SDPE as the primary metric** (first column in every results table), with MAPE and median absolute percentage error reported alongside for completeness.

The *Standard Deviation of Percentage Error* (SDPE) measures *precision*—the tightness of the error distribution—and is defined as the sample standard deviation (with Bessel's correction, denominator $N-1$) of the signed relative error $e_i = (\hat{L}_i - L_i)/L_i$:

$$\text{SDPE} = 100 \cdot \sqrt{\frac{1}{N-1}\sum_{i=1}^{N}(e_i - \bar{e})^2}, \qquad \bar{e} = \frac{1}{N}\sum_{i=1}^{N} e_i.$$

We accompany each SDPE with a 95% bootstrap confidence interval (1000 resamples, random seed 42). The *Mean Absolute Percentage Error* (MAPE) measures accuracy: $\text{MAPE} = \frac{1}{N}\sum |e_i| \times 100$. A precise estimator with a small constant offset can be recalibrated; an imprecise one cannot, so SDPE is the primary quality signal.

5.1 Benchmark Models

We compared GART 2.0 against six baseline estimators representing the evolution of TSP approximation theory. To ensure reproducibility, we present the exact mathematical formulations used in our Python implementation, with constants derived directly from the source code.

5.1.1 Beardwood-Halton-Hammersley (BHH) Theorem

The foundational asymptotic bound for TSP assumes the tour cost scales with the hypervolume of the vertex set [Beardwood et al., 1959]. Our implementation estimates the tour cost L as:

$$\hat{L}_{\text{BHH}} = \beta_d \cdot n^{\frac{d-1}{d}} \cdot V^{\frac{1}{d}} \tag{5}$$

where V is the volume of the convex hull (or bounding box in high dimensions) and β_d is a dimension-dependent constant. For $d = 2$ we use the large-scale empirical estimate $\beta_2 = 0.7124 \pm 0.0002$ of Johnson et al. [1996], obtained from Concorde-verified optima on uniform random point sets up to $n = 316{,}228$. A separate statistical-mechanics analysis by Percus and Martin [1996] reports $\beta_3 \approx 0.6978$ for the 3D case; we do not apply BHH for $d \geq 3$ because the hull-to-bounding-box volume ratio decays exponentially with dimension, making V unstable as an estimator [Carlsson and Yu, 2024].

5.1.2 Cavdar–Sokol (CS) Distribution-Free Estimator

To handle anisotropic distributions, Cavdar and Sokol [2015] proposed a distribution-free estimator that replaces simple volume metrics with statistical moments of coordinate deviations. This estimator is restricted to 2D:

$$\hat{L}_{\text{CS}} = \alpha_1 \sqrt{n \cdot \sigma'_x \sigma'_y} + \alpha_2 \sqrt{\frac{n \cdot A \cdot \sigma_x \sigma_y}{\bar{c}_x \bar{c}_y}} \tag{6}$$

where A is the area of the convex hull, σ_k is the standard deviation of coordinates in dimension k, $\bar{c}_k$ is the mean absolute deviation from the centroid, and σ'_k is the standard deviation of these absolute deviations. The regression coefficients are fitted by Cavdar and Sokol [2015] as $\alpha_1 = 2.791$ and $\alpha_2 = 0.2669$. Because the model was trained on axis-aligned rectangular graphs, applying it to arbitrarily oriented point sets requires a preprocessing step: coordinates are rotated to align with the minimum-area bounding rectangle [Freeman and Shapira, 1975], following the procedure described in Section 5 of Cavdar and Sokol [2015]. All dispersion statistics are then computed on the rotated coordinates.

5.1.3 Chien Estimator

Chien [1992] proposed a probabilistic estimator for logistics route planning that scales tour cost with the square root of the number of stops and the service area:

$$\hat{L}_{\text{Chien}} = k_1 \sqrt{n \cdot A} + k_2 \frac{n}{p} \tag{7}$$

where A is the convex hull area, n is the number of stops, p is the number of routes, and k_1, k_2 are empirically fitted constants ($k_1 = 0.98$, $k_2 = 0$, $p = 1$ for a single tour). Like BHH, this estimator depends on convex hull area and is restricted to 2D.

5.1.4 Hilbert Space-Filling Curve

This constructive heuristic maps d-dimensional points onto a 1-dimensional interval $[0, 1]$ using a fractal Hilbert curve [Bartholdi and Platzman, 1982]. The points are sorted by their Hilbert index $h(x)$, and the tour is constructed by visiting vertices in this sorted order:

$$\hat{L}_{\text{Hilbert}} = \sum_{i=1}^{n-1} \|x_{(i)} - x_{(i+1)}\|_2 + \|x_{(n)} - x_{(1)}\|_2 \tag{8}$$

This approach preserves locality without constructing a full distance matrix, offering $O(n \log n)$ complexity.

5.1.5 MST Ratio

The control baseline scales the weight of the Minimum Spanning Tree by a dimension-dependent factor $\rho(d)$ derived from empirical observation of the MST-to-TSP ratio [Percus and Martin, 1996]:

$$\hat{L}_{\text{MST-Ratio}} = \rho(d) \cdot L_{\text{MST}} \tag{9}$$

For 2D instances, we utilize $\rho(2) = 1.075$, calibrated on uniform random instances [Percus and Martin, 1996]. For higher dimensions, we use $\rho(3) = 1.05$ and $\rho(d) = 1.0 + 0.075 \cdot (2/d)$ for $d \geq 4$, reflecting the known convergence of the MST-to-TSP ratio as dimensionality increases [Percus and Martin, 1996]. These constants were calibrated for uniform distributions; the elevated SDPE observed for MST Ratio in Table 8 reflects the inability of a fixed constant to adapt to the topological variation present in non-uniform test instances.

5.2 Datasets

We evaluate on three datasets: a multi-dimensional synthetic set, a 2D diverse synthetic set, and the EUC_2D subset of TSPLIB95. The synthetic sets are held-out test splits from our generation pipeline (Appendix B); TSPLIB95 is real-world. The evaluation pipeline assumes symmetric metric distances; instances that violate the triangle inequality (e.g. the bridge-graph outlier `brg180`) are excluded from every aggregate.

5.2.1 Multi-dimensional synthetic ($d = 2$–100)

16,907 held-out instances with $n \in [5, 1000]$ and $d \in \{2, 3, \ldots, 50, 100\}$, sampled by stratified dimension–size–grid buckets across 16 per-dimension distribution types drawn from the generators used for our training set: uniform, normal, clustered (k-means [Lloyd, 1982]), and autoregressive-correlated. Stratification enforces zero overlap with the training split. $d = 100$ is outside the training range and serves as the extrapolation test.

5.2.2 2D diverse synthetic

2,580 instances, $d = 2$, $n \in [5, 1000]$, coordinate grid side-length $G \in [1000, 10000]$. Five distribution classes (Table 6) each probe a different assumption made by classical 2D estimators. Four classes (isotropic, biased, geometric, clustered) follow conventions used in the TSP-instance-generation literature, including the variants reported by Cavdar and Sokol [2015] for the distribution-free baseline. The fifth class, *Line Noise*, generates near-linear manifolds of the form $y = mx + b + \epsilon$ with small ϵ. This geometry is *not* represented in the training distribution: we include it precisely to measure how the model handles point sets that collapse toward a 1D manifold, where hull-based formulas vanish and bounding-box formulas over-predict.

Table 6: 2D benchmark dataset composition (2,580 instances).

Class	**Generators**	**Count**	**Stress target**
Isotropic Noise	Uniform, Normal, Triangular, Truncated Exp.	840	Control: $A_{\text{hull}} \approx A_{\text{box}}$.
Biased / Skewed	Squeezed, Uni-Tri, Tri-Squeezed, Correlated	840	Aspect-ratio distortion: $\sigma_x \neq \sigma_y$.
Geometric Struct.	Grid, Boundary (hollow), X-Central	630	Lattice regularity, hollow voids.
Clustered	k-Means ($k \in [5, 20]$)	60	Bimodal edge-weight distribution.
Line Noise†	Linear manifold $y = mx + b + \epsilon$	210	Hull/box divergence as data collapses toward 1D.
Total		**2,580**	

† Not represented in training; included to measure generalization to unseen geometries.

5.2.3 TSPLIB95 EUC_2D common subset

78 EUC_2D instances with n from 14 to 85,900 [Reinelt, 1991]. This is the intersection where every benchmark model is directly applicable: all classical baselines require 2D Euclidean coordinates, and the MST Ratio baseline needs only a distance matrix. The remaining 32 metric TSPLIB95 instances (CEIL_2D, ATT pseudo-Euclidean, GEO great-circle, EXPLICIT-metric) are not natively Euclidean and are evaluated in Section 6 via MDS embedding, which only GART 2.0 supports. For large instances where the dense $n \times n$ distance matrix would exceed memory, we replace it with a Delaunay-based MST construction [Delaunay, 1934] that uses only $O(n)$ edges.

5.3 Results

Every table below reports seven quantities per model row: instance count N, MAPE, SDPE, median $|\%\,\text{error}|$, R^2 on tour cost (coefficient of determination, upper-bounded by 1, unbounded below), r_α = Pearson correlation between predicted and true $\alpha = L_{\text{TSP}}/L_{\text{MST}}$ (bounded $[-1, 1]$, undefined for constant predictors such as MST Ratio), and mean total prediction time in milliseconds (feature extraction + MST + inference).

Each bucket closes with a separate "*Optimal solver*" row whose only populated cell is the Time column. We define the **optimal solver time** as the shorter of the two wall-clock times from locally running CONCORDE [Applegate et al., 2006] and LKH-3 [Helsgaun, 2017] on a single core, without any timeout, where both solvers report the same tour length (and that tour length matches the published TSPLIB optimum where applicable). When only one solver reaches the verified optimum, that solver's time is used; when neither is locally tractable (very large n), the bucket is annotated with a citation to published benchmark times. The column reports milliseconds in scientific notation so the six orders of magnitude spanning small-n and large-n instances remain legible and directly comparable to the model times in the same column.

5.3.1 Multi-dimensional benchmark

We present the 16,907-instance multi-dimensional benchmark in two orthogonal slices: by instance size (Table 7) and by dimension (Table 8). Both tables summarize the same underlying data.

Table 7: ND benchmark by instance size (16,907 instances). Same data as Table 8. *Optimal solver* row = mean CONCORDE/LKH-3 wall time per bucket.

n bucket	Model	N	MAPE (%)	SDPE (%)	Median (%)	R^2	r_α	Time (ms)
$[5, 10]$	GART 2.0	4,227	**1.36**	**1.90**	0.99	1.000	0.971	211
	MST Ratio	4,227	18.89	5.44	17.89	0.966	—	13.3
	Hilbert	4,227	8.11	7.45	5.75	0.995	—	31.3
	Optimal solver	—	—	—	—	—	—	1.6×10^2 ms
$[11, 50]$	GART 2.0	2,816	**0.88**	**1.17**	0.73	1.000	0.965	235
	MST Ratio	2,816	7.45	2.89	6.94	0.996	—	34.5
	Hilbert	2,816	19.59	12.35	15.60	0.981	—	75.7
	Optimal solver	—	—	—	—	—	—	1.2×10^2 ms
$[51, 100]$	GART 2.0	3,522	**0.66**	**0.79**	0.58	1.000	0.979	271
	MST Ratio	3,522	5.33	2.13	4.80	0.998	—	56.8
	Hilbert	3,522	24.38	14.10	20.18	0.973	—	147
	Optimal solver	—	—	—	—	—	—	1.0×10^2 ms
$[101, 200]$	GART 2.0	703	**0.55**	**0.63**	0.47	1.000	0.986	338
	MST Ratio	703	4.44	1.74	4.12	0.999	—	95.0
	Hilbert	703	27.03	13.60	23.40	0.965	—	311
	Optimal solver	—	—	—	—	—	—	3.7×10^2 ms
$[201, 500]$	GART 2.0	2,115	**0.50**	**0.56**	0.39	1.000	0.987	388
	MST Ratio	2,115	4.06	1.50	3.80	0.999	—	152
	Hilbert	2,115	28.26	12.75	26.22	0.960	—	477
	Optimal solver	—	—	—	—	—	—	1.4×10^3 ms
$[501, 1000]$	GART 2.0	3,524	**0.48**	**0.54**	0.31	1.000	0.986	570
	MST Ratio	3,524	3.82	1.32	3.63	0.999	—	426
	Hilbert	3,524	29.97	12.77	28.71	0.953	—	2056
	Optimal solver	—	—	—	—	—	—	6.9×10^3 ms
TOTAL	**GART 2.0**	**16,907**	**0.81**	**1.17**	**0.63**	**1.000**	**0.989**	**330**
	MST Ratio	16,907	8.56	6.89	5.61	0.999	—	133
	Hilbert	16,907	21.28	14.55	16.22	0.962	—	552
	Optimal solver	—	—	—	—	—	—	1.7×10^3 ms

GART 2.0 reaches 0.81% MAPE and 1.17% SDPE overall, against 8.56% / 6.89% for MST Ratio and 21.28% / 14.55% for Hilbert. Precision tightens monotonically as n grows (SDPE drops from 1.90% at $n \leq 10$ to 0.55% at $n \geq 500$) and as d grows (2.75% at $d = 2$ down to 0.50% at $d = 100$), consistent with the theoretical convergence $\alpha \to 1$ in high dimensions [Percus and Martin, 1996]. MST Ratio's SDPE is nearly flat because its fixed constant does not adapt to instance structure. Hilbert degrades with n because space-filling locality breaks on non-uniform distributions.

Table 8: ND benchmark by dimension. Same 16,907 instances as Table 7. $d = 100$ is outside the training range (extrapolation). *Optimal solver* row shows mean CONCORDE/LKH-3 wall time.

d bucket	Model	N	MAPE (%)	SDPE (%)	Median (%)	R^2	r_α	Time (ms)
$d = 2$	GART 2.0	647	**1.82**	**2.75**	0.99	1.000	0.977	317
	MST Ratio	647	13.19	10.69	8.95	0.995	—	98.0
	Hilbert	647	30.36	16.40	35.31	0.804	—	82.8
	Optimal solver	—	—	—	—	—	—	6.3×10^3 ms
$d = 3$–5	GART 2.0	1,941	**1.16**	**1.83**	0.64	1.000	0.981	319
	MST Ratio	1,941	11.72	8.06	8.02	0.996	—	97.6
	Hilbert	1,941	35.53	15.82	39.96	0.737	—	105
	Optimal solver	—	—	—	—	—	—	2.8×10^3 ms
$d = 6$–10	GART 2.0	3,240	**0.83**	**1.28**	0.48	1.000	0.986	323
	MST Ratio	3,240	10.27	6.64	7.19	0.997	—	103
	Hilbert	3,240	32.78	14.34	36.20	0.789	—	157
	Optimal solver	—	—	—	—	—	—	1.8×10^3 ms
$d = 15$–25	GART 2.0	1,944	**0.55**	**0.84**	0.31	1.000	0.992	328
	MST Ratio	1,944	8.72	6.01	5.70	0.998	—	114
	Hilbert	1,944	24.73	11.81	26.33	0.846	—	295
	Optimal solver	—	—	—	—	—	—	1.5×10^3 ms
$d = 30$–50	GART 2.0	3,236	**0.36**	**0.58**	0.20	1.000	0.996	330
	MST Ratio	3,236	7.63	5.90	4.60	0.999	—	127
	Hilbert	3,236	17.14	7.98	18.13	0.924	—	494
	Optimal solver	—	—	—	—	—	—	1.2×10^3 ms
$d = 100^*$	GART 2.0	5,899	**0.90**	**0.50**	0.89	1.000	0.997	339
	MST Ratio	5,899	6.54	5.93	3.43	0.999	—	173
	Hilbert	5,899	10.40	4.60	11.08	0.971	—	1084
	Optimal solver	—	—	—	—	—	—	1.2×10^3 ms
TOTAL	**GART 2.0**	**16,907**	**0.81**	**1.17**	**0.63**	**1.000**	**0.989**	**330**
	MST Ratio	16,907	8.56	6.89	5.61	0.999	—	133
	Hilbert	16,907	21.28	14.55	16.22	0.962	—	552
	Optimal solver	—	—	—	—	—	—	1.7×10^3 ms

* $d = 100$ is outside the training range; included to test extrapolation.

5.3.2 2D diverse benchmark

Table 9 disaggregates the 2,580-instance 2D benchmark by instance size.

GART 2.0 leads at 3.23% MAPE / 4.94% SDPE overall. MST Ratio is second at 12.45% / 11.06%. Cavdar, BHH, and Chien exceed 30% SDPE because their enclosure-based formulas amplify variance on the Biased and Line Noise classes, where bounding-box and hull volumes diverge. Hilbert has intermediate MAPE (31%) but lower SDPE (14%): its errors are large but consistent — a systematic space-filling bias. GART's per-bucket r_α stays in the 0.83–0.94 range across all size buckets, confirming that predicted α tracks true α tightly regardless of instance size.

5.3.3 TSPLIB95 EUC_2D benchmark

Table 10 reports per-bucket results across all 78 EUC_2D instances, with n from 14 to 85,900. Bucket boundaries are chosen to equalize N across the four buckets (23/16/20/19 instances) rather than to fall on round-number n values.

Real-world transfer is summarized in Table 10: GART 2.0 has the tightest SDPE (the primary metric) in every size bucket, with confidence intervals reported in the table. The largest instance, $n = 85{,}900$, lies well beyond the training range and is predicted in under half a second. At large n the true α concentrates near the asymptotic BHH value, so a

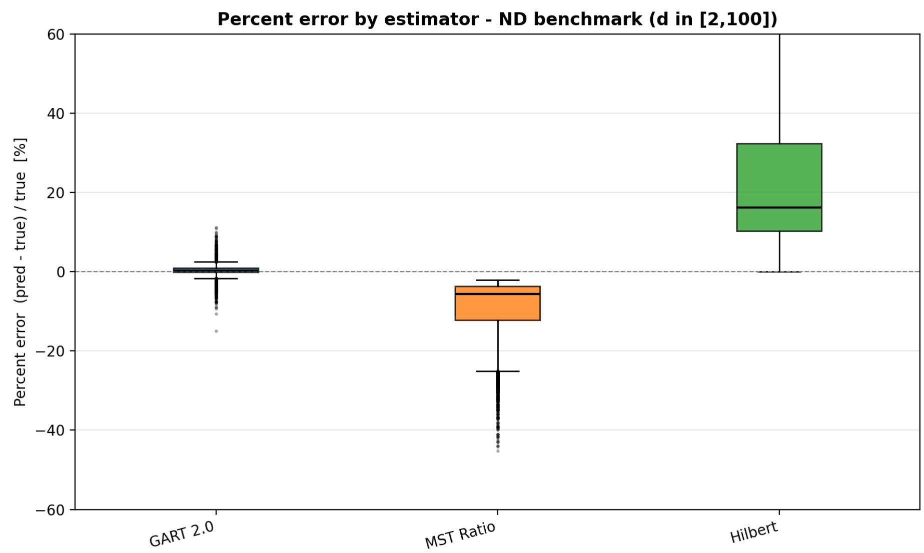

Figure 1: Percent-error distribution per model on the ND benchmark (16,907 instances).

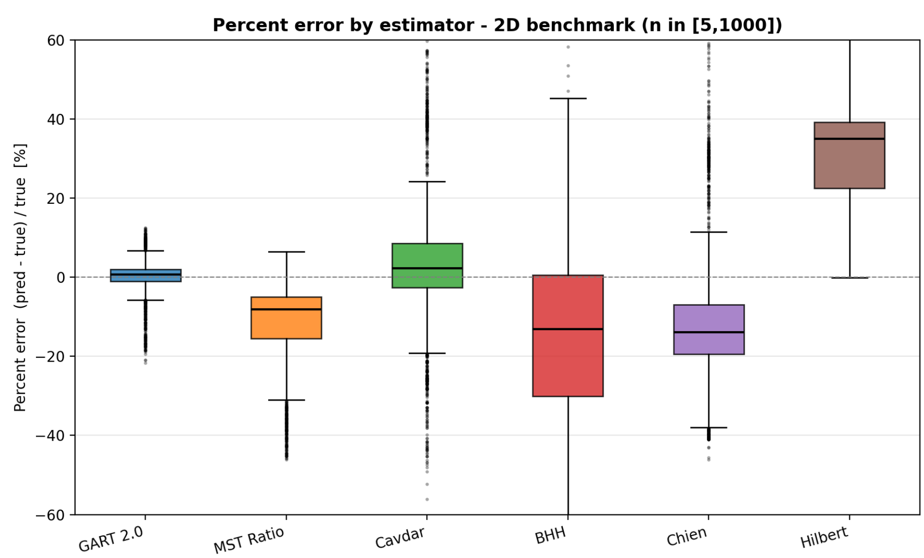

Figure 2: Percent-error distribution per model on the 2D diverse benchmark (2,580 instances).

fixed MST-ratio constant remains competitive on *accuracy* while still losing on *precision*. Classical baselines BHH, Cavdar, and Chien all degrade past $n = 400$ as real-world instances deviate from the uniform-density assumptions their derivations require.

5.4 Discussion

Three patterns recur across benchmarks.

MST Ratio converges to GART 2.0 at high density. In the $d = 100$ ND bucket, MST Ratio MAPE is 6.54% vs GART 0.90%; in the largest TSPLIB bucket the two estimators agree within 1% MAPE. The reason is that $\alpha \to 1$ as dimension or density grows [Percus and Martin, 1996], so any fixed constant near 1 becomes a decent approximation.

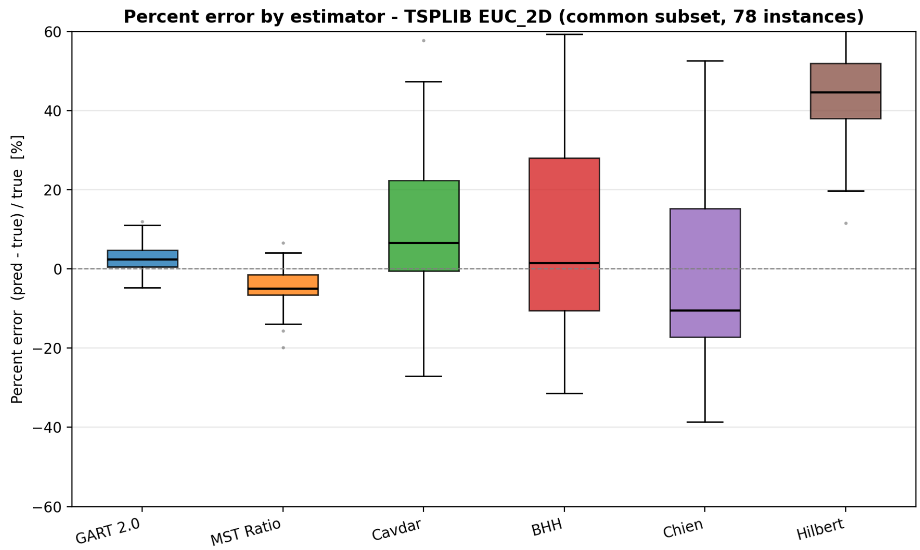

Figure 3: Percent-error distribution per model on the TSPLIB95 EUC_2D subset (78 instances).

GART's advantage lives in the mid-density regime where α still varies meaningfully with instance topology — exactly the regime that dominates practical logistics.

Geometric estimators are inapplicable in $d \geq 3$. BHH, Cavdar, and Chien all require the convex hull area or bounding-box volume. Hull computation costs $O(n^{\lfloor d/2 \rfloor})$ [O'Rourke, 1998] and the hull-to-box volume ratio shrinks exponentially with d [Carlsson and Yu, 2024]; these baselines are absent from the ND results by construction.

Cost structure. On the TSPLIB log — the only timing log in our pipeline that cleanly separates preparation from inference — GART 2.0 spends 74.8% of wall time on feature extraction and MST construction and 25.0% on LightGBM inference; classical baselines spend $\geq 99\%$ on feature/MST preparation. GART's overhead relative to a classical baseline is the feature pipeline itself, not the tree ensemble. The pipeline cost is sunk for any MST-based estimator.

Known weak points: grid lattices and near-linear manifolds (Line Noise 10.86% MAPE in Table 9), where MST topology becomes structurally degenerate. These cases are rare in real logistics but informative for mapping the model's boundary.

6 Application: Non-Euclidean Estimation via MDS Embedding

TSPLIB95 contains 32 metric instances that are not native EUC_2D: 4 CEIL_2D (Euclidean with ceiling-rounded distances), 2 ATT (pseudo-Euclidean), 10 GEO (great-circle geographic), and 16 EXPLICIT-metric (distance-matrix only). Classical estimators cannot process these: BHH and Cavdar require 2D coordinates, Chien requires convex hull area, and Hilbert requires a Euclidean embedding. We present GART 2.0 as the sole estimator viable on these cases. Timings are reported as an exploration of capability, not a comparison. We exclude the EXPLICIT outlier `brg180` from every aggregate because its distance matrix has 85,368 triangle-inequality violations: GART 2.0 (like any MST-based estimator) assumes symmetric metric distances, so non-metric instances are out of scope for this model.

6.1 MDS preprocessing

Classical multidimensional scaling [Torgerson, 1952, Borg and Groenen, 2005] embeds an $n \times n$ distance matrix D into k-dimensional Euclidean coordinates. Let $J = I - \frac{1}{n}\mathbf{1}\mathbf{1}^\top$ be the centering matrix and $D^{(2)}$ the elementwise-squared distance matrix. The doubly-centered inner-product matrix is $B = -\frac{1}{2}JD^{(2)}J$; its eigendecomposition $B = V\Lambda V^\top$

yields the embedding $X = V_k \Lambda_k^{1/2}$ on the top k positive eigenvalues [Cox and Cox, 2000]. We pick the smallest k retaining $\geq 99.9\%$ of the eigenvalue variance, capped at $k = 100$. MST features are computed from the *original* distance matrix (preserving metric accuracy), while geometric features use the embedded coordinates. The selected k is passed to the model as the dimension feature. Appendix C gives the derivation.

6.2 Results on non-Euclidean TSPLIB95

GART 2.0 generalizes across all four distance types with 4–7% MAPE. GEO instances achieve the lowest error (4.44%) because great-circle distances admit a low-stress Euclidean embedding at typical geographic scales. EXPLICIT-metric instances reach 5.94% because their matrices may mildly violate the triangle inequality, inflating MDS stress. CEIL_2D runtime is inflated by two of its four instances exceeding $n = 10{,}000$, which dominates the 581 ms mean.

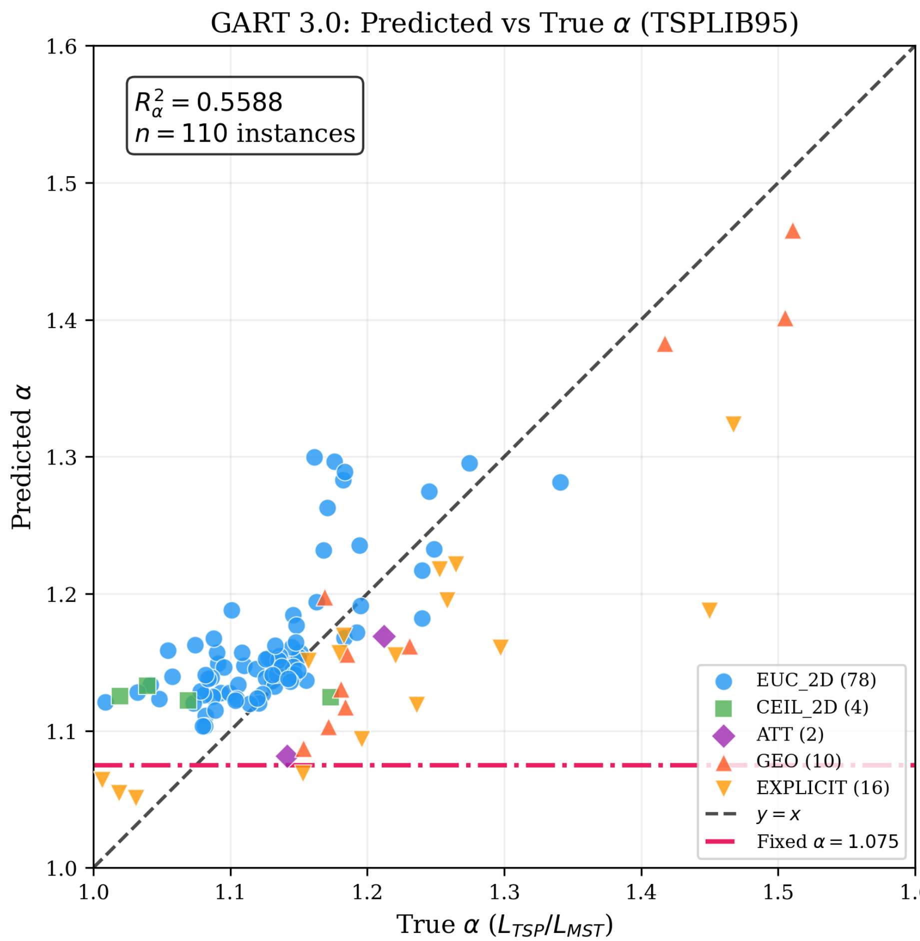

Figure 4: GART 2.0: predicted vs. true $\alpha = L_{\text{TSP}}/L_{\text{MST}}$ on all 110 TSPLIB95 instances, colored by distance type. Combined data from the 78 EUC_2D instances of Section 5.3.3 and the 32 non-Euclidean instances of Table 11.

Figure 4 combines all 110 TSPLIB95 metric instances on a single predicted-versus-true α plot. Predictions cluster along the diagonal across every distance type, showing that the MST-to-TSP ratio is learnable even when coordinates are derived rather than native. A pattern worth noting: the non-Euclidean subsets (GEO, EXPLICIT-metric) produce wider true-α spreads than EUC_2D, so there is more signal for the model to track — GART's r_α reaches 0.97 on GEO and 0.84 on EXPLICIT-metric. On EUC_2D, most real-world instances converge to $\alpha \in [1.07, 1.15]$; that narrow band makes the ranking harder because a small absolute prediction error spans a larger fraction of the true-α range. Instances with higher true-α variation are therefore where MST-based features have the most headroom, and where the next generation of features should focus.

7 Conclusion

We presented GART 2.0, a TSP tour-length estimator that predicts the MST-to-TSP scaling factor α using 29 dimension-agnostic features combining MST topology with lightweight coordinate-range descriptors. Three benchmarks—2,580 synthetic 2D instances, 16,907 multi-dimensional test instances spanning $d = 2$ to $d = 100$, and 110 metric TSPLIB95 instances (78 EUC_2D plus 32 non-Euclidean via MDS)—demonstrate that MST-topology features are sufficient to recover the per-instance scaling factor with high *precision* across dimensions and distribution types, without any enclosure geometry. Precision, measured through SDPE and validated with Levene's test, is the primary contribution: a precise estimator with a small systematic offset can be recalibrated; an imprecise one cannot.

7.1 Contributions

Across the 2D diverse benchmark, the multi-dimensional synthetic set (d up to 100), and the 78 EUC_2D TSPLIB95 instances (with n up to 85,900, many times the training cap), GART 2.0 is materially tighter in precision than the MST-ratio baseline, as summarised by the SDPE columns and their 95% bootstrap confidence intervals in the benchmarking tables. On the 32 non-Euclidean TSPLIB95 instances (CEIL_2D, ATT, GEO, EXPLICIT-metric), to our knowledge GART 2.0 is the first tour-length estimator evaluated across every TSPLIB95 metric distance type through a single MDS-hybrid pipeline. The Delaunay-accelerated feature pipeline yields sub-500 ms prediction at $n = 1{,}000$.

7.2 Limitations and Future Work

We identify three specific limitations of GART 2.0 as delivered here, each pointing at a concrete next step.

1. **The α regressor is not yet tight.** The coefficient of determination on α and related diagnostics (reported in the benchmarking tables) show that a non-trivial share of the variance in the target is still unexplained. The current feature set is a pragmatic first cut; a more expressive regressor (deeper LightGBM, a small neural head on top of the current features, or a probabilistic model targeting the conditional distribution of α rather than its mean) should reduce SDPE further without requiring new feature engineering.
2. **Dimension-tolerant, not dimension-agnostic.** The feature set applies to arbitrary d, but feature computation time is not uniform across d. Pairwise distances, centroid statistics, and MST construction all scale with d, and the TSPLIB benchmarking tables show that per-instance wall time grows roughly with d even when accuracy does not degrade. In that sense GART 2.0 generalises *correctness* to high dimensions but not *cost*; a principled dimension-reduction front-end (learned embedding, random projection, or adaptive feature sampling) would make the claim of dimension-independence operational rather than only statistical.
3. **There is a regime where the MST-ratio baseline is already good enough.** At large n under near-uniform spatial density, the true α concentrates tightly around its asymptotic BHH value. In this regime a constant-α estimator is comparable in accuracy to GART 2.0 while costing less to evaluate. GART 2.0's precision advantage is strongest for lower n and for distributions that deviate from uniform (clustered, anisotropic, non-Euclidean via MDS); a natural follow-on is a dispatch layer that runs the cheap MST-ratio baseline when the instance is detected to be in the easy regime and defers to GART 2.0 otherwise.

The natural next step for *use* of the estimator is to embed it as a cost oracle inside a larger optimization loop: a Vehicle Routing Problem sub-tour estimator [Mariescu-Istodor and Fränti, 2021], or a bounding function in branch-and-bound. Both settings would measure whether GART 2.0's precision translates into faster, more reliable end-to-end decisions.

Table 9: 2D diverse benchmark by instance size (2,580 instances). The *Optimal solver* row shows mean wall-clock time of the ground-truth solver per bucket (CONCORDE or LKH-3, whichever produced the training tour).

n bucket	Model	N	MAPE (%)	SDPE (%)	Median (%)	R^2	r_α	Time (ms)
$[5, 10]$	GART 2.0	360	**4.53**	**5.52**	3.80	0.993	0.936	132
	MST Ratio	360	28.95	9.98	29.17	0.785	—	9.84
	Cavdar	360	17.37	14.23	13.18	0.904	—	6.67
	BHH	360	59.32	9.72	57.70	0.203	—	7.81
	Chien	360	25.81	7.22	26.24	0.842	—	1.14
	Hilbert	360	8.35	8.83	5.79	0.965	—	3.63
	Optimal solver	—	—	—	—	—	—	1.8×10^2 ms
$[11, 50]$	GART 2.0	480	**4.18**	**5.86**	2.77	0.994	0.925	133
	MST Ratio	480	15.16	9.65	13.63	0.949	—	15.2
	Cavdar	480	12.73	16.26	8.45	0.943	—	8.85
	BHH	480	28.38	9.27	27.64	0.817	—	8.94
	Chien	480	19.64	18.66	17.88	0.888	—	1.29
	Hilbert	480	27.18	10.36	26.95	0.804	—	3.04
	Optimal solver	—	—	—	—	—	—	1.2×10^2 ms
$[51, 100]$	GART 2.0	630	**3.57**	**5.53**	2.05	0.995	0.915	173
	MST Ratio	630	11.45	8.59	9.34	0.975	—	29.4
	Cavdar	630	19.07	29.37	7.45	0.897	—	8.59
	BHH	630	16.22	13.07	14.98	0.912	—	11.0
	Chien	630	23.38	30.71	16.69	0.853	—	0.56
	Hilbert	630	34.23	9.81	34.08	0.734	—	3.87
	Optimal solver	—	—	—	—	—	—	4.3×10^2 ms
$[101, 200]$	GART 2.0	120	**2.86**	**5.01**	1.30	0.996	0.929	196
	MST Ratio	120	8.31	7.03	6.72	0.989	—	32.7
	Cavdar	120	18.70	40.86	3.36	0.911	—	7.86
	BHH	120	11.82	20.14	6.85	0.946	—	10.2
	Chien	120	25.75	42.76	15.23	0.853	—	0.30
	Hilbert	120	36.61	10.44	36.48	0.717	—	7.39
	Optimal solver	—	—	—	—	—	—	1.8×10^3 ms
$[201, 500]$	GART 2.0	380	**2.48**	**4.21**	1.14	0.997	0.897	215
	MST Ratio	380	7.19	6.04	5.94	0.992	—	82.9
	Cavdar	380	26.11	55.53	3.21	0.867	—	14.3
	BHH	380	15.35	32.00	4.26	0.928	—	9.72
	Chien	380	30.90	55.96	14.49	0.823	—	5.74
	Hilbert	380	38.59	9.93	38.21	0.717	—	39.0
	Optimal solver	—	—	—	—	—	—	6.1×10^3 ms
$[501, 1000]$	GART 2.0	610	**1.92**	**3.16**	0.95	0.997	0.912	424
	MST Ratio	610	5.71	4.54	5.00	0.994	—	237
	Cavdar	610	23.93	58.49	3.39	0.872	—	25.3
	BHH	610	14.75	33.62	5.08	0.931	—	15.9
	Chien	610	29.90	59.29	14.07	0.823	—	14.0
	Hilbert	610	38.24	11.16	38.21	0.728	—	93.9
	Optimal solver	—	—	—	—	—	—	3.5×10^4 ms
TOTAL	**GART 2.0**	**2,580**	**3.23**	**4.94**	**1.73**	**0.998**	**0.937**	**226**
	MST Ratio	2,580	12.45	11.05	8.16	0.992	—	81.1
	Cavdar	2,580	19.82	42.03	6.13	0.909	—	13.1
	BHH	2,580	23.82	32.04	15.55	0.943	—	11.1
	Chien	2,580	25.78	42.55	17.25	0.874	—	4.71
	Hilbert	2,580	31.01	14.23	34.99	0.801	—	30.3
	Optimal solver	—	—	—	—	—	—	9.5×10^3 ms

Table 10: TSPLIB95 EUC_2D benchmark by instance size (78 instances, four balanced buckets). SDPE is primary (first numeric column) and includes a 95% bootstrap CI (1000 resamples, seed 42). GART 2.0 rows are bolded. *Optimal solver* row shows mean CONCORDE wall time in milliseconds.

n bucket	Model	SDPE (%) [95% CI]	MAPE (%)	Median (%)	R^2	r_α	Time (ms)	N
$[51, 150]$	**GART 2.0**	**3.28** [2.03, 4.04]	**2.83**	1.71	0.996	0.704	9.33	23
	MST Ratio	4.05 [2.70, 4.84]	6.93	6.12	0.978	—	0.93	23
	Cavdar	14.37 [7.77, 19.01]	10.00	6.44	0.936	—	0.71	23
	BHH	15.99 [7.69, 20.89]	16.09	13.59	0.958	—	0.73	23
	Chien	14.58 [6.61, 18.92]	18.45	17.84	0.948	—	0.11	23
	Hilbert	11.46 [8.20, 13.91]	39.91	38.92	0.758	—	0.72	23
	Optimal solver	—	—	—	—	—	854	—
$[151, 400]$	**GART 2.0**	**3.23** [2.00, 4.11]	**2.89**	2.50	0.997	0.841	18.4	16
	MST Ratio	5.25 [2.45, 7.17]	5.94	4.93	0.986	—	4.45	16
	Cavdar	26.84 [10.56, 34.22]	19.60	11.29	0.577	—	0.66	16
	BHH	23.29 [11.57, 29.59]	16.06	6.53	0.793	—	0.87	16
	Chien	25.90 [13.29, 31.96]	21.22	16.22	0.860	—	0.14	16
	Hilbert	7.27 [3.45, 10.25]	44.39	42.42	0.596	—	1.61	16
	Optimal solver	—	—	—	—	—	15,525	—
$[401, 1500]$	**GART 2.0**	**3.92** [2.55, 4.84]	**3.60**	2.25	0.999	0.666	34.5	20
	MST Ratio	3.51 [2.58, 4.14]	4.85	5.65	0.993	—	101.7	20
	Cavdar	36.92 [19.56, 45.96]	28.84	14.19	0.825	—	1.11	20
	BHH	49.16 [20.33, 64.83]	34.04	14.70	0.831	—	1.57	20
	Chien	44.38 [20.70, 57.82]	30.75	18.33	0.943	—	0.43	20
	Hilbert	10.81 [7.24, 13.35]	48.57	46.95	0.278	—	5.99	20
	Optimal solver	—	—	—	—	—	4,541,959	—
$n > 1500$	**GART 2.0**	**3.28** [2.05, 3.96]	**4.47**	3.50	1.000	0.486	87.5	19
	MST Ratio	3.52 [2.29, 4.40]	2.93	2.21	0.998	—	539.9	19
	Cavdar	53.76 [15.47, 78.40]	35.25	18.58	0.821	—	2.64	19
	BHH	50.47 [15.98, 74.50]	34.45	25.11	0.872	—	2.82	19
	Chien	44.16 [15.34, 65.12]	26.16	13.70	0.930	—	1.80	19
	Hilbert	14.47 [9.42, 19.08]	48.76	49.11	0.597	—	36.0	19
	Optimal solver	—	—	—	—	—	1,341,641,880	—
Total	**GART 2.0**	**3.52** [2.95, 4.04]	**3.44**	2.37	1.000	0.727	36.7	78
	MST Ratio	4.54 [3.54, 5.36]	5.22	4.96	0.998	—	158.8	78
	Cavdar	36.30 [22.72, 48.46]	22.95	11.44	0.832	—	1.28	78
	BHH	40.75 [25.39, 53.04]	25.16	13.41	0.880	—	1.48	78
	Chien	35.83 [23.46, 46.44]	24.05	16.84	0.934	—	0.61	78
	Hilbert	11.84 [9.83, 13.62]	45.20	44.60	0.623	—	10.9	78
	Optimal solver	—	—	—	—	—	591,595,588	—

Table 11: GART 2.0 on non-Euclidean TSPLIB95 (32 instances). r_α = Pearson correlation; undefined when $N < 3$ (ATT). Embedded dimension k chosen by MDS variance retention. CONCORDE is inapplicable to non-Euclidean distance types; solver time is unavailable.

Distance type	N	MAPE (%)	SDPE (%)	Median (%)	r_α	Time (ms)	Embed. k (avg [range])
CEIL_2D	4	7.11	5.66	6.96	-0.323	581	2 [2,2]
ATT	2	4.40	0.83	4.40	—	16.6	53 [6,100]
GEO	10	4.44	2.61	4.95	0.971	16.5	16 [2,61]
EXPLICIT-metric	16	5.94	5.85	5.19	0.838	12.3	37 [8,100]
TOTAL	**32**	**5.52**	**5.74**	**5.12**	**0.857**	**84.9**	—

References

Richa Agarwala, David L Applegate, Donna Maglott, Gregory D Schuler, and Alejandro A Schäffer. A fast and scalable radiation hybrid map construction and integration strategy. *Genome Research*, 10(3):350–364, 2000.

Takuya Akiba, Shotaro Sano, Toshihiko Yanase, Takeru Ohta, and Masanori Koyama. Optuna: A next-generation hyperparameter optimization framework. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pages 2623–2631. ACM, 2019. doi: 10.1145/3292500.3330701.

David L. Applegate, Robert E. Bixby, Vašek Chvátal, and William J. Cook. *The Traveling Salesman Problem: A Computational Study*. Princeton University Press, 2006.

John J. Bartholdi, III and Loren K. Platzman. An O(N log N) planar travelling salesman heuristic based on spacefilling curves. *Operations Research Letters*, 1(4):121–125, 1982. doi: 10.1016/0167-6377(82)90034-6.

Jillian Beardwood, John H. Halton, and John Michael Hammersley. The shortest path through many points. *Mathematical Proceedings of the Cambridge Philosophical Society*, 55:299–327, 1959. doi: 10.1017/S0305004100034095.

Ingwer Borg and Patrick J.F. Groenen. *Modern Multidimensional Scaling: Theory and Applications*. Springer, 2nd edition, 2005. doi: 10.1007/0-387-28981-X.

John Gunnar Carlsson and Keyi Yu. A new upper bound for the Euclidean TSP constant. *INFORMS Journal on Computing*, 2024. doi: 10.1287/ijoc.2024.0538.

Bahar Cavdar and Joel Sokol. A distribution-free tsp tour length estimation model for random graphs. *European Journal of Operational Research*, 243(2):588–598, 2015. doi: 10.1016/j.ejor.2014.12.020.

T William Chien. Operational estimators for the length of a traveling salesman tour. *Computers & Operations Research*, 19(6):469–478, 1992. doi: 10.1016/0305-0548(92)90002-M.

Nicos Christofides. Worst-case analysis of a new heuristic for the travelling salesman problem. Technical Report 388, Carnegie-Mellon University, 1976. Reprinted in Operations Research Forum, 3:20, 2022.

William Cook and André Rohe. Computing minimum-weight perfect matchings. *INFORMS Journal on Computing*, 11 (2):138–148, 1999.

Trevor F. Cox and Michael A.A. Cox. *Multidimensional Scaling*. Chapman and Hall/CRC, 2nd edition, 2000. ISBN 9781584880943.

Boris Delaunay. Sur la sphère vide. *Bulletin de l'Académie des Sciences de l'URSS, Classe des Sciences Mathématiques et Naturelles*, 6:793–800, 1934.

Doratha E. Drake and Stefan Hougardy. A simple approximation algorithm for the weighted matching problem. *Information Processing Letters*, 85(4):211–213, February 2003. doi: 10.1016/S0020-0190(02)00393-9.

Ran Duan and Seth Pettie. Linear-time approximation for maximum weight matching. *Journal of the ACM (JACM)*, 61 (1):1–23, 2014.

Finmile. Revolutionizing last-mile delivery for unparalleled efficiency and customer satisfaction. Whitepaper, Finmile, 2025. URL `https://finmile.co/resources/finmile-revolutionizing-last-mile-delivery`.

Herbert Freeman and Ruth Shapira. Determining the minimum-area encasing rectangle for an arbitrary closed curve. *Communications of the ACM*, 18(7):409–413, 1975. doi: 10.1145/360881.360919.

Russell Halper and Srinivasa Raghavan. The mobile facility routing problem. *Transportation Science*, 45(3):413–434, 2011. doi: 10.1287/trsc.1100.0335.

Michael Held and Richard M. Karp. A dynamic programming approach to sequencing problems. *Journal of the Society for Industrial and Applied Mathematics*, 10(1):196–210, 1962. doi: 10.1137/0110015.

Keld Helsgaun. An extension of the lin-kernighan-helsgaun tsp solver for constrained traveling salesman and vehicle routing problems. *Roskilde University, Tech. Rep*, 2017.

Chuck Holland, Jack Levis, Ranganath Nuggehalli, Bob Santilli, and Jeff Winters. Ups optimizes delivery routes. *Interfaces*, 47(1):8–23, 2017. doi: 10.1287/inte.2016.0875. Winner of the 2016 Franz Edelman Award.

David S. Johnson, Lyle A. McGeoch, and Edward E. Rothberg. Asymptotic experimental analysis for the Held–Karp traveling salesman bound. In *Proceedings of the 7th Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*, pages 341–350, 1996.

I.T. Jolliffe. *Principal Component Analysis*. Springer, 2002.

Anna R. Karlin, Nathan Klein, and Shayan Oveis Gharan. A (slightly) improved approximation algorithm for metric TSP. In *Proceedings of the 53rd Annual ACM Symposium on Theory of Computing*, pages 32–45, 2021. doi: 10.1145/3406325.3451009.

Richard M. Karp. Reducibility among combinatorial problems. In Raymond E. Miller and James W. Thatcher, editors, *Complexity of Computer Computations*, pages 85–103. Plenum Press, 1972. doi: 10.1007/978-1-4684-2001-2_9.

Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. Lightgbm: A highly efficient gradient boosting decision tree. In *Advances in Neural Information Processing Systems*, pages 3146–3154, 2017.

Suixian Kou, Bruce Golden, and Stefan Poikonen. Optimal TSP tour length estimation using standard deviation as a predictor. *Computers & Operations Research*, 148:105993, 2022. doi: 10.1016/j.cor.2022.105993.

Siu Kwan Lam, Antoine Pitrou, and Stanley Seibert. Numba: A LLVM-based Python JIT compiler. In *Proceedings of the Second Workshop on the LLVM Compiler Infrastructure in HPC*, pages 1–6, 2015. doi: 10.1145/2833157.2833162.

Stuart Lloyd. Least squares quantization in pcm. *IEEE Transactions on Information Theory*, 28(2):129–137, 1982.

Scott M. Lundberg and Su-In Lee. A unified approach to interpreting model predictions. In *Advances in Neural Information Processing Systems 30 (NeurIPS)*, pages 4765–4774, 2017.

Radu Mariescu-Istodor and Pasi Fränti. Solving the large-scale TSP problem in 1 h: Santa Claus Challenge 2020. *Frontiers in Robotics and AI*, 8:689908, 2021. doi: 10.3389/frobt.2021.689908.

Karl Menger. Untersuchungen über allgemeine metrik. *Mathematische Annalen*, 100(1):75–163, 1928.

Joseph O'Rourke. *Computational Geometry in C*. Cambridge University Press, 2nd edition, 1998.

Xuanhao Pan, Yan Jin, Yuandong Ding, Mingxiao Feng, Li Zhao, Lei Song, and Jiang Bian. H-TSP: Hierarchically solving the large-scale traveling salesman problem. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 37, pages 9345–9353, Washington, DC, USA, June 2023. AAAI Press. doi: 10.1609/aaai.v37i8.26120.

Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, et al. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.

Allon G. Percus and Olivier C. Martin. Finite-size corrections to the Euclidean traveling salesman problem. *Physical Review Letters*, 76(8):1188–1191, 1996. doi: 10.1103/PhysRevLett.76.1188.

Franco P. Preparata and Se June Hong. Convex hulls of finite sets of points in two and three dimensions. *Communications of the ACM*, 20(2):87–93, 1977.

Robert Clay Prim. Shortest connection networks and some generalizations. *Bell System Technical Journal*, 36(6): 1389–1401, 1957.

Gerhard Reinelt. TSPLIB—a traveling salesman problem library. *ORSA Journal on Computing*, 3(4):376–384, 1991. doi: 10.1287/ijoc.3.4.376.

Kate Smith-Miles and Jano van Hemert. Towards objective measures of algorithm performance across instance space. *Computers & Operations Research*, 37(5):850–862, 2010.

Warren S. Torgerson. Multidimensional scaling: I. Theory and method. *Psychometrika*, 17(4):401–419, 1952. doi: 10.1007/BF02288916.

Taha Varol, Okan Örsan Özener, and Erinc Albey. Neural network estimators for optimal tour lengths of traveling salesperson problem instances with arbitrary node distributions. *Transportation Science*, 58(1):45–66, 2024. doi: 10.1287/trsc.2022.0015.

Pauli Virtanen, Ralf Gommers, Travis E. Oliphant, Matt Haberland, Tyler Reddy, David Cournapeau, Evgeni Burovski, Pearu Peterson, Warren Weckesser, Jonathan Bright, et al. SciPy 1.0: fundamental algorithms for scientific computing in Python. *Nature Methods*, 17:261–272, 2020. doi: 10.1038/s41592-019-0686-2.

Guiling Wang, Guohong Cao, Thomas La Porta, and Wensheng Zhang. Mobile sink routing for data collection in wireless sensor networks. *IEEE Transactions on Mobile Computing*, 7(6):756–770, 2008. doi: 10.1109/TMC.2007.70734.

A Code and Data Availability

Our implementation, trained model weights, and benchmark scripts are available at `https://github.com/sfeldmanMIG25/Area-and-Distribution-Free-Estimator-for-TSP`. Synthetic instances can be regenerated using the provided scripts with fixed random seeds.

B Training Data Generation Details

The training dataset was constructed by generating synthetic TSP instances across four distribution classes, with dimensions independently assigned per axis:

- **Uniform**: Coordinates drawn from $\mathcal{U}(0, G)$ where G is the grid size.
- **Normal (Gaussian)**: Coordinates drawn from $\mathcal{N}(G/2, G/6)$, truncated to $[0, G]$.
- **Clustered (*k*-means)**: $k \in [3, 15]$ cluster centers placed uniformly, with points drawn from $\mathcal{N}(\mu_k, \sigma_k)$ around each center.
- **Correlated**: Autoregressive structure across dimensions: $x_{d+1} = \rho \cdot x_d + \epsilon$, producing axis-aligned anisotropy.

Each instance's dimensions are independently assigned a distribution class, producing anisotropic configurations where topology varies by axis. Node counts span $n \in [20, 1{,}000]$ with dimensions $d \in \{2, 3, 4, 5, 6, 7, 8, 9, 10, 15, 20, 25, 30, 35, 40, 45, 50\}$ (17 dimension values). Ground-truth solutions were computed by running both Concorde and LKH-3, retaining the shorter result. Solved instances exist for d up to 100 (85,229 total across 18 dimension values); the $d = 100$ instances are reserved as the extrapolation bucket in the ND benchmark (Section 5.3.1).

The 51,818 instances are partitioned into training (72%), validation (18%), and test (10%) splits using stratified sampling by dimension and size bracket.

C MDS Embedding for Non-Euclidean Instances

For TSPLIB instances with non-Euclidean distance types (GEO: geographic coordinates with great-circle distances; EXPLICIT: distance matrix only), we apply classical Multidimensional Scaling (MDS) to obtain a Euclidean coordinate representation.

Given an $n \times n$ distance matrix D, classical MDS computes the eigendecomposition of the doubly-centered matrix $B = -\frac{1}{2} J D^{(2)} J$ where $J = I - \frac{1}{n}\mathbf{1}\mathbf{1}^T$ and $D^{(2)}$ is the element-wise squared distance matrix. The embedding coordinates are $X = V_k \Lambda_k^{1/2}$ where V_k and Λ_k are the top k eigenvectors and eigenvalues.

We embed into k dimensions, selecting the smallest k that retains $\geq$99.9% of eigenvalue variance (capped at $k = 100$). For GEO instances, the selected dimension ranges from 2 to 61; for EXPLICIT instances, from 8 to 100. The MST features (edge weights, degrees, diameter) are computed from the *original* distance matrix to preserve metric accuracy, while the geometric features (centroid statistics, bounding volume, aspect ratio) use the MDS embedding. The dimension k is passed to the model as the `dimension` feature.

For GEO instances, the distance function is well-approximated by Euclidean distances at moderate scales, yielding low MDS stress and 4.44% MAPE. For EXPLICIT instances, the distance matrices may violate metric properties (triangle inequality), producing higher stress. After excluding `brg180`—a bridge graph whose distance matrix is fundamentally non-metric (MST cost exceeds $2\times$ the optimal tour)—the remaining 16 EXPLICIT instances achieve 5.94% MAPE.